

Enhancing Membership Inference Attacks in Federated Learning Based on Overfitting Property

Xiaoyan Yin^{1,†}, *Member, IEEE*, Yangyang Li^{2,†}, Congrui Bai¹, Qingjie Han³ and Yanjiao Chen^{4,*}, *Senior Member, IEEE*

¹ School of Information Science and Technology, Northwest University, Xi'an, Shaanxi, China
(Email: yinxy@nwu.edu.cn, 202333100@stumail.nwu.edu.cn).

² Academy of Cyber, Beijing, China (Email: liyangyang@live.com).

³ Glodon Software Co., Ltd, Xi'an, China (Email: hanqj@glodon.com).

⁴ College of Electrical Engineering, Zhejiang University, Hangzhou, China (Email: chenyanjiao@zju.edu.cn).

[†] Co-first author, these authors contributed equally to this work.

* Correspondence: chenyanjiao@zju.edu.cn.

Abstract— Membership inference attacks have been proposed to infer whether a specific sample is in the training dataset of a victim model. Inferred membership may reveal sensitive information, e.g., personal health condition deduced from a disease prediction model. In federated learning where local datasets of different participants are supposed to be protected, membership inference attacks may still pose a privacy threat. However, existing membership inference attacks in federated learning select the final epoch and random intermediate epochs during training to solicit features for the attack, which may lead to poor attack performance.

In this paper, we propose a novel membership inference attack in federated learning, which attempt to find the key epoch during training that is most indicative of the differences between member and non-member samples. Inspired by the overfitting phenomena that often occurs during the training of learning models, we derive the key epoch as the onset of overfitting. We design efficient algorithms to pinpoint the key epoch and leverage attention mechanism to weight the importance of different features. We evaluate our scheme on four datasets and compare experimental results with state-of-the-art attack algorithms. Experimental results demonstrate that our attack outperforms existing works in terms of inference accuracy.

Index Terms—Federated learning, membership Inference attacks, overfitting, attention mechanism

I. INTRODUCTION

Federated learning (FL) is a form of distributed machine learning that aims to allow multiple participants to learn a global model together under the control of a central server without sharing individual data, thereby protecting the data privacy [1]. However, federated learning, like traditional machine learning model, is still vulnerable to various privacy attacks [2], [3], [4]. Membership inference attack (MIA) tries to determine whether a given sample is in the training set of the target victim model [5], which will leak personal private information.

Existing works have demonstrated that a model's vulnerability to MIA is closely related to its generalization error [6]. During training, the model is likely to overfit on the training samples, i.e., the model *remembers* more about the training samples. An overfitted model has larger generalization

error, and is more likely to exhibit different characteristics on training samples and non-training samples. Shokri et al. [5] showed that the model's output has distinguishable properties about training and non-training data, which could be exploited by the attack model. Existing works have shown that if members and non-members have significantly different outputs on the target model, the attack model will have a higher inference accuracy. Generally, training and non-training data have significant output on the last epoch of the target model. Therefore, an attack model can be trained to judge whether the target data is a member or not based on mainly the information provided in the last epoch [7]. However, for non-training data samples with similar output as the training samples from the victim model, MIA is degraded to random guessing. This motivates us to increase the attack model's capability of differentiating member and non-member samples.

We are inspired by the common phenomenon in machine learning, i.e., overfitting [7]. Due to overfitting, the more thoroughly the target model is trained on the training set, the more membership information the target model will remember. Therefore, we can select the features of the target model that reveal the most membership information to train the attack model [8]. Based on this observation, we propose a new membership inference attack against federated learning scenarios. The main idea is that when the model starts to overfit, the training set and the non-training set begin to perform greatly different, i.e., the prediction accuracy on the training set and the non-training set diverges. Specifically, the accuracy of the target model on non-training samples will stop increasing or even drop. For members and non-members with similar performance at the last epoch, their performance will be distinctive at such a key epoch. For these data, using the features at the key epoch will greatly improve the membership inference accuracy. However, various data have different dependencies on the key epoch and the last epoch. Luo et al. [9] carried out risk prediction for diseases by applying a time-aware attention network, which once again proved the reliability of deep learning in risk prediction. Inspired by this, we creatively add the attention mechanism to

our membership inference attack to assign different weights to the two epochs according to the characteristics of different samples.

In this paper, we make the following contributions.

- We propose a new membership inference attack in federate learning based on the model's overfitting property. Our proposed membership inference attack performs better in distinguishing non-training samples that have similar prediction performance to training samples on the final global model.
- We design an efficient algorithm to pinpoint the key epoch when the overfitting occurs. The features from the key epoch and the final epoch are combined using the attention mechanism to help infer membership of different types of samples.
- We conduct comprehensive experiments to verify that our proposed attack can improve the inference accuracy compared with existing works.

II. METHODOLOGY

In this section, we first analyze the drawbacks of existing attacks, based on which we present our attack method. The attack overview is shown in Fig. 1.

A. Motivation

In federated learning, we consider that the attacker is a participant, who targets at inferring membership information of other participants. The attacker can observe and obtain the updated global model parameters at each epoch. Given a target sample $(\mathbf{x}, y)$, $\mathbf{x}$ is the sample's feature, and y is the true label. The available attack information for attackers is as follows. (1) The attacker runs the global model f on the target input $\mathbf{x}$, and obtains all the hidden layers' output $h(\mathbf{x})$ and the model's output $f(\mathbf{x})$. (2) Based on the model's output $f(\mathbf{x})$ and the label y , the attacker can compute the loss function $L(f(\mathbf{x}), y)$ in a forward pass. (3) The attacker can also compute the gradients of the loss with respect to each layer's parameters in back propagation. In summary, the available features are label, loss, output and gradients. As a participant, the attacker can observe these features at each epoch. The attacker can stack the results of multiple rounds. For example, after N rounds, the loss feature can be constructed as $L = \{L_1, \dots, L_N\}$.

Existing membership inference attacks usually use the information of several random epochs to construct a collective feature as input into the attack model. However, the random selection of epochs may yield redundant and less useful information.

Existing attacks usually use manually-selected features. For example, a sample in the training set is expected to have a high prediction confidence, and its prediction confidence vector is expected to be close to one-hot encoding. Therefore, the cross entropy of the confidence score is relatively low. In comparison, for a non-member sample, its cross entropy should be relatively high. Loss function value is also often used by existing methods to differentiate member and non-member samples. Given sample $(\mathbf{x}, y)$, its loss function value on the target model is $L(f(\mathbf{x}), y)$. The lower the loss value,

the more likely that the sample is a member. Although these distinctions can be learned by the attack model to determine whether a sample is a member, there are many non-member samples that also have high prediction confidence and low loss values, making the attack model less accurate.

To address the above two problems, we first propose a mechanism to select the key epochs that generate the most useful information, and then design an attention mechanism to focus on the most important features for membership inference.

B. Key Epoch Selection

We determine the key epoch as the epoch at which overfitting occurs in the global model. Overfitting is closely related to membership inference attacks. A model with a larger overfitting degree also has a larger generalization error and is more likely to leak membership information. The generalization error is defined as

$$R_{\text{gen}}(A, D, S, L) = \mathbb{E}_{z \sim S} [L(A, z)] - \mathbb{E}_{z \sim D} [L(A, z)], \quad (1)$$

where A is the target model, D is training set, S is test set, L is the loss function of target model.

An overfitted model's prediction accuracy on the training set is greater than that on non-training dataset, resulting in the model containing abundant membership information. Therefore, we determine the key epoch based on the characteristics of overfitting. When overfitting happens, as the training epoch increases, the training set performs progressively better than the non-training set on the target model. More specifically, after overfitting, the prediction accuracy on the training set will keep increasing, while the prediction accuracy on the non-training set is expected to drop. If a sample is not a member in the training set and if the sample can be accurately predicted by the target model at the last epoch, then this sample can also be correctly predicted by the target model at the key epoch with a high probability. The prediction confidence of the sample at the two epochs will be close, and the difference of loss values between these two epochs will also be relatively small. However, for samples in the training set, the difference between the loss values of these two epochs will be large, and the difference between the prediction confidences on the target models of the key epoch and the last epoch is larger than that of non-member data. Our goal is to find the epoch with such distinguishing power, denote it as K . We will use the attention mechanism to assign weights to the features of epoch K and the last epoch.

The attacker, who is a participant, can calculate the accuracy of the global model on his test set at each epoch during training. The key epoch is when the global model's performance stabilizes on the test set. Therefore, we calculate the prediction accuracy gap between two consecutive epochs. If the accuracy gap decreases, we choose the current epoch as the key epoch. We formulate the process of finding epoch K as

$$\arg \max_K (\Delta Tr - \Delta Te), \quad (2)$$

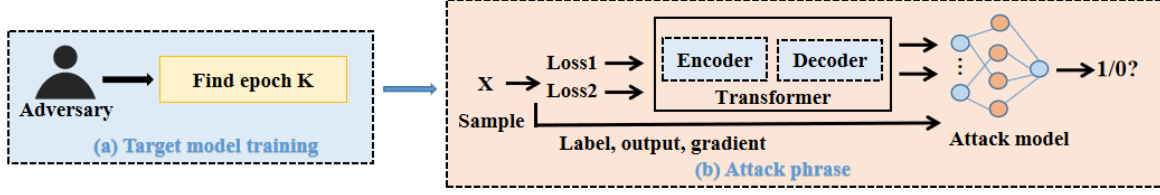


Fig. 1: Overview of MIA. After an attacker uses the algorithm to find epoch K , we assign weights to epoch K and last epoch's loss value using the Transformer model consisting of an encoder and a decoder. Then we take them and the rest of the features as input to the attacking model.

where K represents the key epoch, ΔTr denotes the loss value gap between the training set at epoch K and the last epoch, and ΔTe is the gap on non-training set. Therefore, the larger the $\Delta Tr - \Delta Te$, the more different the performance of members and non-members in this epoch. ΔTr and ΔTe can be calculated as

$$\begin{aligned}\Delta Tr &= \mathbb{E}_{z \sim D} [L(A, z)] - \mathbb{E}_{z \sim l^D} [L(A, z)], \\ \Delta Te &= \mathbb{E}_{z \sim S} [L(A, z)] - \mathbb{E}_{z \sim l^S} [L(A, z)],\end{aligned}\quad (3)$$

where l represents the last epoch, $\mathbb{E}_{z \sim l^D} [L(A, z)]$ is the loss value at the last epoch on the training set on target model, $\mathbb{E}_{z \sim D} [L(A, z)]$ is the loss value at epoch K on the training set, $\mathbb{E}_{z \sim l^S} [L(A, z)]$ is the loss value at the last epoch of non-training set, and $\mathbb{E}_{z \sim S} [L(A, z)]$ is the loss value at epoch K of the non-training set.

The algorithm for finding epoch K is presented in Algorithm 1. Note that the key epoch corresponds to the epoch where the accuracy of the test set on the target model starts to decline. It implies that the key round will only appear after several rounds of training, i.e., not the first few epochs. However, the accuracy of the test set on the target model fluctuates before stabilization in the process of learning. Therefore, to pinpoint the key epoch, we need to take into account the fluctuation at the beginning of training (lines 3-17). Lines 18-26 perform the task of finding the key epoch K afterwards.

Toy example. We give an example of addressing accuracy fluctuations of the test set on the target model at the beginning of training. Without loss of generality, we assume that the accuracy of the test set on the target model in the first six epochs are $\mathcal{A}_0(S) = 0.4$, $\mathcal{A}_1(S) = 0.42$, $\mathcal{A}_2(S) = 0.45$, $\mathcal{A}_3(S) = 0.32$, $\mathcal{A}_4(S) = 0.4$, $\mathcal{A}_5(S) = 0.45$, respectively. In the first iteration through the loop in lines 3-17, we have $m1 = 0.4$, $m2 = 0.82$, and $m0 = 0.82$. In the second iteration through the loop in lines 3-17, we $m1 = 0.45$, $m2 = 0.77$, $m0 = 0.82$, and $J = 3$, thus the loop terminates.

Algorithm 1 Find Epoch K

Input: Train dataset D , Test dataset S , Target model A , Accuracy function $\mathcal{A}$, Loss function L , Total training epochs l , Training epoch $i, j, J \in [0, l]$.

Output: Epoch K

```

1:  $m0 \leftarrow 0, m1 \leftarrow 0, m2 \leftarrow 0, J \leftarrow 0$ ,
2:  $\Delta Tr \leftarrow 0, \Delta Te \leftarrow 0, \Delta \leftarrow 0$ .
3: for  $j = 0; j < l/2; j++$  do
4:   if  $j \% 2 == 0$  then
5:      $m1 \leftarrow \mathcal{A}_j(S)$ .
6:   end if
7:   if  $j \% 2 == 1$  then
8:      $m2 \leftarrow m1 + \mathcal{A}_j(S)$ .
9:     if  $m0 < m2$  then
10:       $m0 = m2$ .
11:    else
12:       $J = j$ .
13:    break
14:   end if
15: end if
16: end for
17: return  $J$ 
18: for  $i = J; i < l; i++$  do
19:    $\Delta Tr \leftarrow \mathbb{E}_{z \sim D} [L(A, z)] - \mathbb{E}_{z \sim l^D} [L(A, z)]$ .
20:    $\Delta Te \leftarrow \mathbb{E}_{z \sim S} [L(A, z)] - \mathbb{E}_{z \sim l^S} [L(A, z)]$ .
21:   if  $\Delta < \Delta Tr - \Delta Te$  then
22:      $\Delta \leftarrow \Delta Tr - \Delta Te$ .
23:      $K = i$ .
24:   end if
25: end for
26: return  $K$ 

```

C. Attention Mechanism

Given a sample x , if the loss value of x at the last epoch is relatively large, there is no need to use epoch K since x can be regarded as a non-member with high probability. If the loss value of x is relatively small, x may be a member or a non-member. In this case, adding epoch K can further determine whether the data is a member. To treat the above two cases differently, we use the attention mechanism to pay attention to the required epoch depending on the current judgment.

Specifically, we use a transformer model to assign corresponding weights to the two epochs. Among the loss value, prediction vector, gradients and true label at each epoch, the loss value is the most important feature. So we use the transformer model to assign weights to the two epochs' loss value, and then multiply the weights obtained by the loss value and send them to the attack model.

Given the input loss value matrix $L = [L_1, L_2]$, we use a standard one-layer transformer model (denoted as T) to learn the dependencies between L_1 and L_2 , containing an encoder and a decoder.

$$h = [h_1, h_2] = T([L_1, L_2]), \quad (4)$$

where h_1, h_2 is the hidden representation of each loss value.

Then we calculate the attention score using self-attention layer in the transformer.

$$\eta_t = \mathbf{W}_\eta^\top \mathbf{h}_t + b_\eta, \quad (5)$$

$$\eta = [\eta_1, \eta_2], \quad (6)$$

where $\mathbf{W}_\eta$ and b_η are parameters to be learned, and $t = 1, 2$.

We use a softmax layer to generate attention weights for L_1 and L_2 as

$$\alpha = \text{Softmax}(\eta) = [\alpha_1, \alpha_2]. \quad (7)$$

Before sending the loss values of the two epochs to the attack model, we use the attention weights to multiply the loss values.

$$L_t = L_t * \alpha_t. \quad (8)$$

Finally, after processing the input of the attack model, we design a loss function for the attack model.

$$\min[\sum_{z \sim D} (\mathcal{A}(z) - 1)^2 + \sum_{z \sim S} (\mathcal{A}(z))^2]. \quad (9)$$

III. EXPERIMENTAL SETUP

In this section, we first describe the datasets, target models, and experimental setup that we use for evaluation, then we demonstrate the significance of epoch K and the attention mechanism in MIA. Finally, we describe the scenarios that are most beneficial to the attacker.

In this paper, we use four datasets for fair experiments: two standard image recognition benchmark datasets CIFAR10, CIFAR100, and Purchase100, Texas100, which are often used in previous work [6, 7, 10].

Target Model. We choose the state-of-the-art Neural Networks architectures as the target models and use TensorFlow, a widely used deep learning framework in academia, to implement our evaluation.

For CIFAR10 and CIFAR100 datasets, we use a standard convolutional neural network Alexnet with five convolution layers, 3 max-pooling layers, a fully connected layer, and a SoftMax layer. We set the dropout rate to 0.3 and use the Adam optimizer with the learning rate of 0.0001 to train the target models, and the CIFAR10's maximum epoch of training is 120, and the CIFAR100's is 700.

For the Purchase100 dataset, we use a fully connected model with layer sizes of 1024, 512, 256, 128, and 100. We use SoftMax as the activation function for the last layer, other layers use the Tanh activation function, and Adam optimizer with a learning rate of 0.001 to learn the model. The maximum epoch of training is 220.

Furthermore, for Texas100, we use the fully connected model like the Purchase100 dataset. We train the model with the Adam optimizer with a learning rate of 0.001. The maximum epoch of training is 700.

We set up the federated learning scenario with a central server and 5 participants except the CIFAR100 dataset (4 participants). The attacker is one of the participants who participate at each epoch of the target model's training. We use the same training dataset sizes for all participants, and each party's training data is uniformly randomly selected from our available dataset.

Attack Model. For loss value, prediction output, and true label, we use the FCN (the fully connected network) component to extract features. Moreover, for gradients, we use a CNN component to extract features. After extracting the corresponding features, the features are fed into an encoder consisting of fully connected layers. The encoder will output the final attack result. To train the attack model, we use the Adam optimizer with a learning rate of 0.001. We initialize the weights using a normal distribution with a mean of 0 and a standard deviation of 0.01. The bias value of all layers is initialized with 0. The batch size of all experiments is 32. We train attack models for 100 epochs and pick the model with the highest test accuracy across all the 100 epochs. We run each experiment for 5 times on each dataset and report the average result.

IV. EVALUATION

A. Existing Work's Study

In this section, we conduct experiments to evaluate the existing attack model's effectiveness for members and non-members that behave similarly to the target model. These similar data's average loss value on the target model is shown in Fig. 2. On our four datasets, all data's loss values gradually decrease during the target model's training. However, after certain epochs, the average loss value of non-members is significantly greater than members, which is consistent with the model's overfitting.

To implement our evaluation, we need to filter out members and non-members that the target model can accurately predict, and we use loss value as the selection criterion. As seen in Fig. 2, for CIFAR10, CIFAR100, Purchase100, and Texas100 datasets, the average loss value of samples that can be predicted accurately by the target model is 1.47, 3.81, 3.68, and 3.70, respectively. Therefore, we select members and non-members with loss values between 1.46-1.48, 3.80-3.82, 3.60-3.70, and 3.60-3.90, respectively. Among them, for Purchase100 and Texas100, since the dataset is relatively small, we choose a larger range than the other two datasets.

The attack accuracy is shown in Table. I. Results show that the attack model's effect is close to random guessing

TABLE I: Attack accuracy for similar members and non-members.

Datasets	Member Size	Non-member Size	Attack Accuracy
CIFAR10	2,000	2,000	50.81%
CIFAR100	2,000	2,000	51.69%
Purchase100	1,000	1,000	50.25%
Texas100	600	600	53.66%

(0.5), which means that the attack model cannot accurately distinguish between members and non-members with similar performance just using the last epoch.

B. Impact Of Epoch K

In this section, we evaluate the improvement of our attack model, which creatively adds epoch K of the target model.

In order to show the effect of epoch K , we first select the members and non-members with similar loss values. Fig. 3 shows the average loss value of these similar members and non-members on the target model during training. On the CIFAR10 dataset, the loss value of non-members from the beginning epochs to a certain epoch in the middle training has a relatively large drop and then slowly decreases. However, for members, the loss value decreases slowly from the beginning to the end. This means that for members and non-members, in addition to the last epoch, we can also find features in the training process that can distinguish members and non-members. On this basis, we can use the corresponding equation to find the epoch K with the largest features.

Then we evaluate the attack accuracy of our attack model on the overall dataset and members and non-members with similar performance, respectively. In order to show the importance of epoch K compared with other training epochs, we set the baselines by selecting several epochs evenly from the beginning to the end of the target model's training. Then, we compare the attack accuracy of each epoch with epoch K . On CIFAR10, epoch K is nearly 25, and we train the target model 120 epochs, so we select [15,45,75,100] comparison epochs. On CIFAR100, we choose the epoch [100,200,300,400,500,600]. On Purchase100, epoch K is nearly 90, we train 220 epochs on target model, so we choose [10,50,130,170,210] epochs. And on Texas100, epoch K is nearly 63, we choose the epoch [100,200,300,400,500,600] for comparison.

Fig. 4 shows the attack accuracy on the whole dataset, and Fig. 5 shows the attack accuracy on members and non-members who behave similarly. It can be seen from the two figures that the epoch K we select has the greatest attack accuracy compared with other epochs, especially on members and non-members who behave similarly. This proves that our solution of adding epoch K is effective.

We also evaluate the attack accuracy of the attack model on each class of CIFAR10, which is shown in Fig. 6. It can be seen that the attack accuracy on each class is different, and our attack model can improve the attack accuracy on each class. To understand the difference in attack accuracy in each class,

TABLE II: comparison of attack accuracy with prediction entropy-based attack method.

Datasets	NN-Based	Loss-Based	Entropy-Based
CIFAR10	71.54%	70.30%	70.20%
CIFAR100	81.55%	79.00%	79.60%
Purchase100	63.80%	62.20%	61.50%
Texas100	73.00%	71.90%	71.70%

TABLE III: comparison of attack accuracy with and without using an attention mechanism.

Datasets	Without Attention	With Attention
CIFAR10	71.54%	71.74%
CIFAR100	81.55%	82.70%
Purchase100	63.80%	63.95%
Texas100	73.00%	73.56%

we also study the gradient norms and prediction uncertainty of members and non-members in each class. Fig. 7 shows the gradient norms of the attack model on each class. Fig. 8 shows the prediction uncertainty of the attack model on each class. It can be seen from Fig. 6 that on CIFAR10, the attack accuracy on categories 2, 3, 4, and 5 are the highest, and in Fig. 7, the difference of the gradient norms between the members and non-members on the 3 and 5 categories is the largest. In Fig. 8, the difference in prediction uncertainty between members and non-members in category 2,3,4,5 is the largest. It can be seen that both the gradient norms and the prediction uncertainty can be used to distinguish members and non-members.

C. Comparison With Baselines

For comparison with baselines, we compare our method with the prediction entropy-based method. Then, to eliminate the uncertainty and unfairness of comparing neural networks with non-neural networks, we also use a prediction loss-based attack and compare the results with the prediction entropy. As can be seen from the table II, the neural network-based attack method we use has the largest effectiveness on the four datasets.

D. Impact Of Attention Mechanism

Table III shows the comparison of attack accuracy using and without using the transformer model. It can be seen from the table that using the attention mechanism can improve the attack accuracy on the overall data. This shows that it is necessary and useful to use the attention mechanism to assign corresponding weights to epoch K and the last epoch for different samples. This is because, at the last epoch, the non-members whose performance on the target model is very different from the members can be distinguished by only using the last epoch, and adding epoch K will weaken this feature for these data.

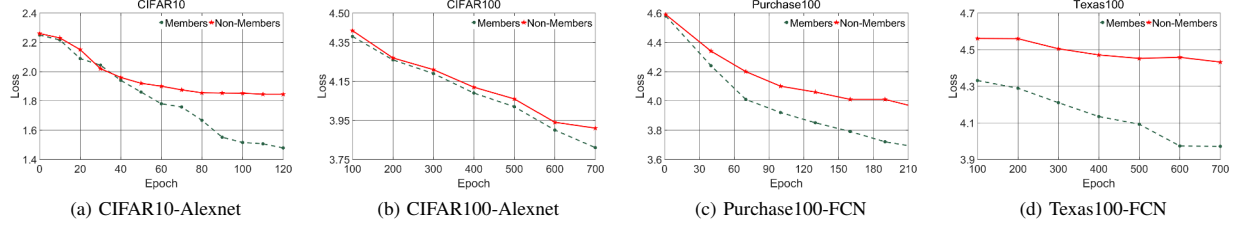


Fig. 2: Average loss value of members and non-members on the overall dataset with the target model's training epochs.

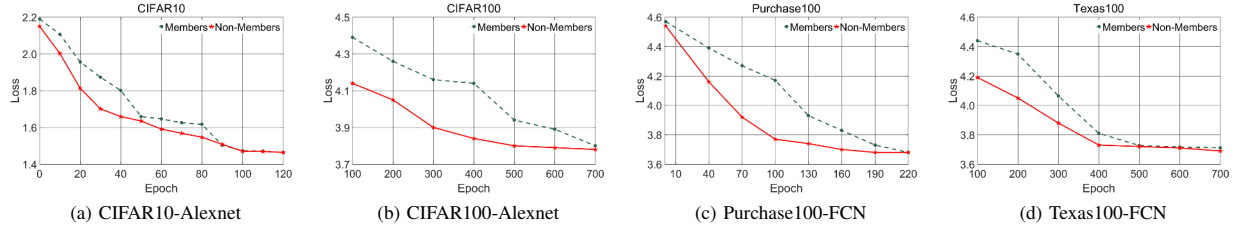


Fig. 3: Average loss value of similar members and non-members with the target model's training epochs on four datasets.

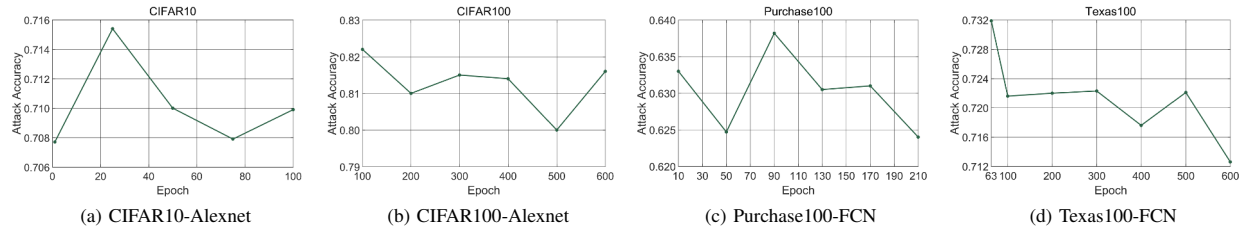


Fig. 4: Attack accuracy on overall datasets and similar members and non-members after adding epoch K . The K is 25, 90, 100, and 65 respectively on CIFAR10, Purchase100, CIFAR100, and Texas100 datasets.

E. The Best Setting For Attacker

When the attacker participates in the federated learning model's training, it can observe and obtain all the parameters of the target global model. It can use the outputs of each layer in the target model, the loss function's gradients over each layer's parameters, and samples' true labels as features to train the attack model. However, if an attacker uses the information from all layers as features to input into the attack model, too many parameters may affect the attack model's efficiency and reduce the training speed. We evaluate the attacker's optimal attack setup in terms of both attack effectiveness and training speed.

To evaluate which layer's output has the biggest impact, we perform the attack separately using each layer's output without using gradients. Table IV shows the attack accuracy on four datasets. It can be seen that the attack accuracy of just using the last layer is close to using all layers' outputs, which means that the output of the target model's last layer contains the most membership information.

To evaluate which layer's gradients have the biggest impact on the attack, we perform the attack separately using the

gradients of each layer without using outputs. Table V shows the attack accuracy on four datasets. The results are similar to the above, and the last layer leaks more membership information. Table VI shows the attack accuracy using different combinations of layers' gradients and outputs on four datasets. The attack accuracy of using all layers is close to that of using only the last layer. Table VII shows the attack accuracy on four datasets when the attack model uses different training sizes. The experimental results show that the attack accuracy will be higher when the attack model is trained with more data.

Therefore, it is best for the attacker to select only the last layer of parameters to get the best attack effect and speed. Furthermore, increasing the attack model's training data will increase the accuracy of the attack.

F. More suitable scenarios

In this section, we describe scenarios that are more beneficial to attackers. From the attacker's perspective, it is not only necessary to improve the accuracy of correctly classifying members as members, but it is also crucial to correctly classify non-members as non-members. In some real-world scenarios,

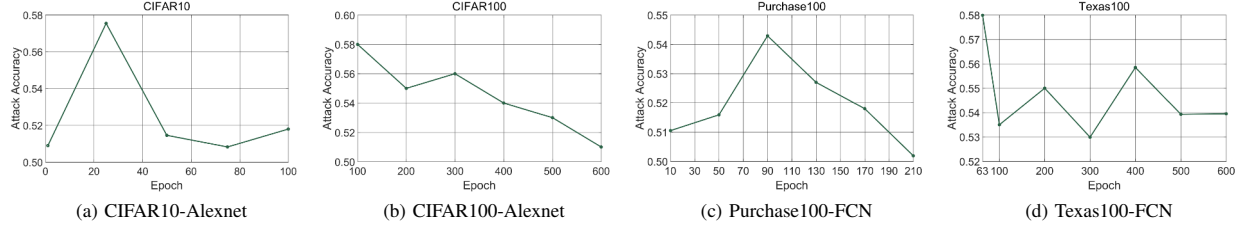


Fig. 5: Attack accuracy on overall datasets and similar members and non-members after adding epoch K . The K is 25, 90, 100, and 65 respectively on CIFAR10, Purchase100, CIFAR100, and Texas100 datasets.

TABLE IV: Attack accuracy of using the outputs of each layer of the target model on four datasets.

Outputs	CIFAR10	CIFAR100	Purchase100	Texas100
Last three layers	68.99%	79.50%	62.17%	71.86%
Last two layers	69.30%	81.01%	62.10%	71.96%
The last layer	69.40%	81.61%	62.14%	72.39%

TABLE V: Attack accuracy of using target model's every layer's gradients on four datasets.

Gradients	CIFAR10	CIFAR100	Purchase100	Texas100
Last three layers	70.29%	78.51%	61.32%	70.66%
Last two layers	70.39%	79.45%	61.62%	70.85%
The last layer	71.52%	81.50%	61.75%	71.06%

TABLE VI: Attack accuracy of using different combinations of layer gradients and outputs on four datasets.

Gradients	Outputs	CIFAR10	CIFAR100	Purchase100	Texas100
Last Layer	—	71.52%	81.45%	61.75%	71.06%
—	Last Layer	69.40%	81.54%	62.14%	72.13%
Last layer	Last layer	71.54%	81.55%	63.80%	73.00%

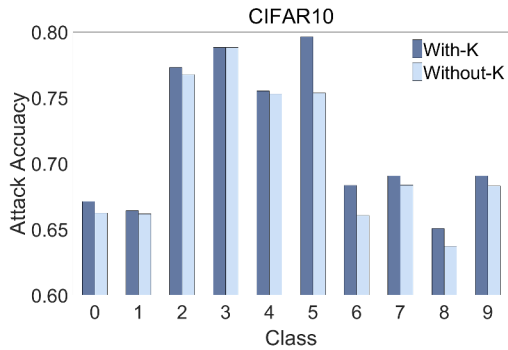


Fig. 6: Attack accuracy of adding and not adding epoch K on CIFAR10's every class.

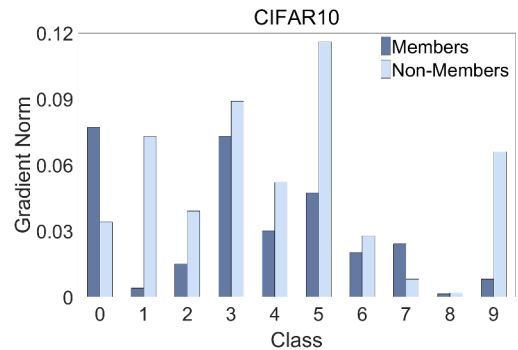


Fig. 7: Average gradient norms distribution of members and non-members on CIFAR10's every class after adding epoch K .

the training data of the target model is limited. However, the untrained data to be applied by the target model is unlimited. Therefore, the attacker's target data is more likely to be non-members, and among these data, there may be non-members that can be accurately predicted by the target model. At this time, its loss value is very similar to that of a well-

trained member, so that it may be misclassified as a member by previous attack methods. Moreover, our attack method can improve the accuracy of the attack model under this circumstance.

The more beneficial scenario we describe is where the

TABLE VII: Attack accuracy for different sizes of the attacker's training dataset on four datasets.

Members Size	Non-members Size	CIFAR10	CIFAR100	Purchase100	Texas100
2,000	2,000	70.90%	79.15%	61.20%	73.00%
4,000	4,000	70.97%	81.10%	63.80%	—
6,000	6,000	71.40%	81.55%	—	—
8,000	8,000	71.54%	81.55%	—	—

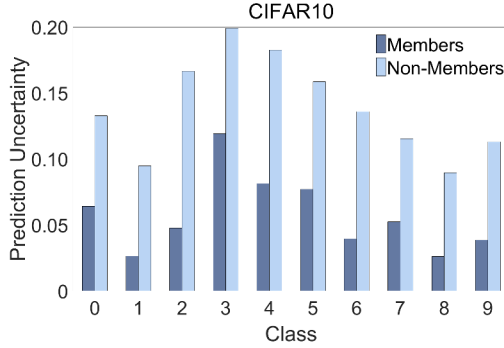


Fig. 8: Average prediction uncertainty distribution of members and non-members on each class of CIFAR10 after adding epoch K .

attacker has a high or larger number of non-members than members, and most of these samples have low confidence scores on the target model. At this point, adding epoch K will greatly improve the accuracy of the attack. This is because, to some extent, our attack model can distinguish between members and non-members that behave similarly. In short, when samples that the attacker wants to judge are really non-members, and their performance on the target model is similar to members, it is more suitable to add epoch K .

V. CONCLUSION

In this paper, to improve the current situation that the existing attack model cannot accurately distinguish between members and non-members with similar performance, we propose a new attack method that allows the attacker to creatively capture a critical epoch K based on the model's overfitting during target model's training. The core of our idea is to use epoch K and the last epoch as the inputs of the attack model. In addition, adding the Transformer model to the attack model can help the attacker dynamically focus on information beneficial to the current attack task. Experimental results have shown that our attack model is superior to existing attack methods and can achieve significant effects in certain scenarios.

REFERENCES

[1] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[2] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, "Beyond inferring class representatives: User-level privacy leakage from federated learning," in *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*. IEEE, 2019, pp. 2512–2520.

[3] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 1322–1333.

[4] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 691–706.

[5] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 3–18.

[6] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 2018, pp. 268–282.

[7] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 739–753.

[8] S. Truex, L. Liu, M. E. Gursoy, L. Yu, and W. Wei, "Demystifying membership inference attacks in machine learning as a service," in *IEEE Transactions on Services Computing*. IEEE, 2019, pp. 1–1.

[9] J. Luo, M. Ye, C. Xiao, and F. Ma, "Hitnet: Hierarchical time-aware attention networks for risk prediction on electronic health records," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 647–656.

[10] G. Ateniese, L. V. Mancini, A. Spognardi, A. Villani, D. Vitali, and G. Felici, "Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers," *International Journal of Security and Networks*, vol. 10, no. 3, pp. 137–150, 2015.